



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

agentní inženýrství, řídicí harness, GitHub App, DarkFactory, automatizace vývoje, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

agentic engineering, harness architecture, GitHub App, DarkFactory, software development automation, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	6
1.1 Motivace a vymezení problému	6
1.2 Cíl práce a výzkumné otázky	6
1.3 Metodika práce	7
2 Teoretická část	8
2.1 Řízení verzí a kontinuální integrace	8
2.2 Jazykové modely a dynamika kontextového okna	9
2.3 Architektura řídicího harnessu a orchestrace	11
2.4 Zapojení člověka do smyčky (Human-in-the-loop)	14
3 Praktická část	16
3.1 Cíl a rozsah systému DarkFactory	16
3.2 Architektura a životní cyklus požadavku	16
3.3 Orchestrační jádro a validační infrastruktura	17
3.4 Systém revizních značek pro lidský dohled nad akademickým textem	18
4 Výsledky a diskuse	20
4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami	20
4.2 Provozní metriky a spolehlivost agentních běhů	21
4.3 Poznatky a systémová úskalí z provozu	21
4.4 Diskuse: Porovnání architektur	21
4.5 Systémová a metodická omezení	22
5 Závěr	23
Seznam příloh	24
A Obsah přiloženého média	25
B Schéma konfiguračního manifestu darkfactory.json	26
C Sdílené workflow pro GitHub Actions	27
D Systémové prompty plánovacího a kódovacího agenta	28
E Protokol revizních značek v sazebním systému Typst	29
Seznam zdrojů	31

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	12
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	17

KONCEPT

1 Úvod

1.1 Motivace a vymezení problému

- **Současný stav automatizace:** Sestavení programu, běh testů, statická analýza i nasazení do produkce jsou dnes plně automatizovány stroji. Úzké hrdlo procesu tvoří samotná modifikace zdrojového kódu — časová prodleva mezi zadáním požadavku a vytvořením otestované změny.
- **Koncept temné továrny (*Dark Factory*):** Průmyslový model plně automatizovaného provozu běžícího bez stálé lidské obsluhy (není nutné svítit). Cílem není vytlačení člověka (inženýr a konstruktér určují záměr a specifikaci), nýbrž úplná eliminace člověka z mechanických, opakujících se úkonů.
- **Limity stávajících nástrojů na bázi LLM:** Většina stávajících řešení (chatovací asistenti, doplňování kódu) řeší pouze izolovaný krok — návrh fragmentu kódu. Chybí ucelená integrace do vývojového cyklu: kontextové zasazení, exekuce v repozitáři, řešení syntaktických regresí a řízení deterministických záruk.
- **Ústřední inženýrská otázka:** Nezkoumáme triviální otázku, zda jazykový model dokáže napsat fragment kódu, nýbrž jaká kontrolní architektura (**harness**) musí model obklopovat, aby jeho výstupům bylo možné v produkčním repozitáři spolehlivě důvěřovat.

 **Návrh na vylepšení: Doporučení k motivaci:** Doporučujeme doplnit srovnávací diagram analogie mezi fyzickou temnou továrnou (materiál → montážní linka → kontrola jakosti → expedice) a agentním vývojem softwaru (GitHub Issue → agentní harness → CI validace → PR/Release). Vizuální metafora výrazně zlepší srozumitelnost pro komisi.

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídicího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíkových rozhodovacích bodech.

Dílčí cíle:

- Vymezit deterministické základy infrastruktury (správu verzí a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct) s detekcí uvíznutí, rozpočtem tahů a bezpečnostním pískovištěm.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

Výzkumné otázky:

- **VO1 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po pull request strukturovat tak, aby role vývojáře spočívala výhradně v architektonickém doзору a schvalování záměru (Human Gate), bez nutnosti ručního psaní rutinního kódu?
- **VO2 (Řízení divergence a spolehlivost smyčky):** Jakými deterministickými mechanismy (stuck detection, rozpočet tahů, circuit breaker) lze v harnessu spolehlivě zabránit patologiím modelu (perseveraci, oscilaci a zacyklení v ReAct smyčce)?
- **VO3 (Integrita paměti a eliminace sémantického posunu):** Jak spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a ztrátě architektonických invariantů při kompresi?

 **Hlubková kritika / Oponentura: Oponentura k výzkumným otázkám:** Otázka VO1 je formulována binárně („Lze vývojový proces strukturovat...“), což svádí k tautologické odpovědi. Rigorózní oponent bude žádat empirické vymezení: Jaké procento rutinních úloh (např. oprava chyby se selhávajícím testem vs. komplexní refaktoring) harness reálně odbaví bez ručního zásahu do kódu? Doporučujeme otázku v obhajobě doplnit o kritérium mezní složitosti úkolu a míry redukce kognitivní zátěže člověka.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Postup sleduje strukturu inženýrského cyklu:

- **1. Konceptní analýza:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Architektonický návrh:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

 **Strukturální upozornění: Chybějící evaluační rámec v metodice:** Metodika práce v současné podobě popisuje inženýrský postup, ale postrádá formální specifikaci evaluačního rámce: definici vzorku úloh pro ověření spolehlivosti (syntetické úlohy vs. reálné bugfixy), stanovení kontrolních metrik (úspěšnost na první pokus, spotřeba tokenů na úspěšný PR) a srovnávací baseline.

2 Teoretická část

2.1 Řízení verzí a kontinuální integrace

Základní deterministická infrastruktura pro autonomní vývoj softwaru staví na systému správy verzí (VCS) a kontinuální integraci (CI):

- **Distribuovaná správa verzí (Git) (1):** Repozitář je formálně orientovaný acyklický graf (*Directed Acyclic Graph*, DAG).
 - **Uzly grafu:** Neměnné objekty revizí (*commits*) provázané kryptografickými hashy (SHA-1 či SHA-256) na své předchůdce.
 - **Větvě grafu:** Pohyblivé ukazatele (*refs*) na konkrétní uzly v historii. Každý přispěvatel i agent disponuje úplnou lokální kopií historie.
- **Izolace stavu ve větvích:** Agent operuje výhradně ve vyhrazené pracovní větvi (`feature/...` či `agent/...`) odbočené z hlavní linie (`main`). Zabraňuje poškození produkčního kódu a kolizím s lidskými vývojáři během experimentování a ladění.
- **Deterministický audit:** Záznam každého kroku agenta jako atomického commitu se strojovými metadaty, časovým otiskem a referencí na zadání. Zaručuje úplnou reprodukovatelnost změn.
- **Strategie slučování (Squash and Merge):** Sloučení celé sekvence pokusných commitů pracovní větve do jediného čistého uzlu. Odstraňuje šum a slepé uličky modelu z hlavní linie projektu.
- **Model pull requestu (PR / Human Gate):** Formalizovaný uzel pro předložení navrženého diffu kódu k revizi. V agentickém vývoji slouží jako hlavní lidská schvalovací brána (*Human Gate*) zobrazující diff, výsledky automatických testů a strukturovaný souhrn změn.
- **Kontinuální integrace (CI) (2):** Automatické sestavení a otestování každé změny v izolovaném prostředí. Plní roli deterministického arbitra správnosti ověřujícího stochastické výstupy jazykového modelu.
- **Požadované kontroly (Required Checks):** Množina automatických úloh (linting, statická typová kontrola, jednotkové a integrační testy), které musí projít úspěšně před povolením sloučení.
 - **Pravidlo deterministického zakončení:** Každá kontrola musí skončit explicitním úspěchem či neúspěchem. Úloha, která se tiše přeskočí bez nahlášení výsledku, může chráněnou větev trvale zablokovat.
- **Reprodukovatelné artefakty:** Výstupy úspěšného průchodu integrační pipeline (spustitelné binární balíčky, knihovny, vysázená PDF dokumentace) svázané s konkrétní verzovací značkou (*tagem*) v historii gítu.

 **Hlubková kritika / Oponentura: Nestálost testů (Flaky Tests) jako systémová slepá skvrna:** Text prezentuje kontinuální integraci jako nekompromisního deterministického arbitra správnosti. V praxi však integrační a end-to-end testy běžně trpí stochastickou nestálostí (časování asynchronních operací, síťové prodlevy, race conditions). Pokud model narazí na náhodně selhávající test, ReAct smyčka začne horečně upravovat funkční kód ve snaze vyřešit neexistující defekt, čímž vnese do repozitáře skryté regrese. Pro spolehlivý provoz musí harness obsahovat mechanismy detekce nestálosti (automatický opakovaný běh v čistém prostředí, izolace stavu) a striktně rozlišovat selhání infrastruktury od regrese modelu.

 **Návrh na vylepšení: Řešení divergencí dlouho běžících větví:** Doporučujeme doplnit princip deterministického rebase: pokud se hlavní větev (`main`) během autonomního běhu agenta posune, harness musí před spuštěním finální validační pipeline provést automatický rebase a ověřit, zda nedošlo k syntaktickým či logickým merge konfliktům.

2.2 Jazykové modely a dynamika kontextového okna

 **Metodické vymezení / Rozsah práce: Metodické vymezení a rozsah práce:** Těžištěm této práce není strojové učení, matematická optimalizace vah ani trénování neuronových sítí. Jazykový model vnímáme jako hotovou inferenční komponentu vystupující v roli stochastického kognitivního jádra. Ústředním předmětem zkoumání je **agentní inženýrství** (*agentic engineering*) a **architektura řídicího harnessu** pro autonomní vývoj softwaru. Následující text je proto záměrně zredukován na nezbytné konceptuální minimum potřebné pro pochopení kontextového okna, spotřeby tokenů, degradace pozornosti a rozhraní nástrojů.

Základní principy velkých jazykových modelů z hlediska agentního inženýrství:

- **Velké jazykové modely (LLM) (3):** Dekodérové transformery (*Decoder-only*, např. řady Claude, GPT, LLaMA, DeepSeek). Vystupují jako stochastické autoregresivní generátory: na základě textové historie opakovaně predikují nejpravděpodobnější následující symboly (kód, text, volání nástroje).
- **Tokeny a tokenizace:** Diskrétní celočíselné jednotky textu zpracováváné deterministickým tokenizérem (např. algoritmem BPE).
- **Embedding:** Převod identifikátorů tokenů do vícerozměrného vektorového prostoru vyjadřujícího sémantickou příbuznost pojmů.
- **Tahy (Turns):** Diskrétní kroky výměny informací v agentní smyčce:
 - ▶ **Tah uživatele / prostředí (User/Environment Turn):** Zadání úkolu nebo vnější událost.
 - ▶ **Tah modelu (Model Turn):** Odpověď modelu nebo emitování požadavku na nástroj.
 - ▶ **Tah vykonání nástroje (Tool Execution Turn):** Výsledek operace vrácený harnessu.

- ▶ **KV cache:** Ukládání mezivýpočtů pozornosti klíčů a hodnot pro neměnnou historii tahů; šetří výpočetní čas inference.
 - **Kontextové okno (*Context Window*):** Pevně limitovaná kapacita paměti modelu. Omezeno kvadratickou složitostí mechanismu pozornosti ($O(N^2)$) vzhledem k délce sekvence N) a velikosti paměti grafických akceleratorů.
 - **Jazyková asymetrie tokenizace:** Text v češtině spotřebovává kvůli bohaté flexi a diakritice 2× až 3× více tokenů než angličtina. Doporučení: systémové prompty, technické plány i interní logy harnessu vést v angličtině.
- Správa paměti a limity komprese kontextu:**
- **Kompakce historie (*Compaction*):** Vyzvání modelu k vytvoření syntetického souhrnu dosavadního průběhu sezení pro uvolnění kapacity kontextového okna.
 - **Patologie destruktivní ztrátové komprese:**
 - ▶ **Vymazání deterministických detailů:** Nevratný zánik čísel řádků, signatur privátních funkcí, přesných cest v souborech a doslovných chybových hlášení kompilátoru.
 - ▶ **Ztráta negativních omezení:** Zákazy (neměnit veřejné API, nepřidávat externí knihovny) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.
 - ▶ **Sémantický posun (*Semantic Drift*):** Kaskádová kumulace drobných zkreslení a halucinací při rekurzivním shrnování ($S_{k+1} = f(S_k, \Delta_k)$), vedoucí k rozchodu modelu se skutečným stavem repozitáře.
 - **Alternativy k naivní textové kompresi:**
 - ▶ **Hierarchická epizodická paměť (RAG):** Ukládání doslovné historie tahů do externí databáze a selektivní injekce pouze bezprostředně relevantních fragmentů.
 - ▶ **Persistentní graf stavu projektu (*Project State Graph*):** Udržování explicitního strukturovaného stavu repozitáře (změněné soubory, otevřené úkoly, výsledky testů, invarianty) mimo kontextové okno.
 - ▶ **Selektivní prořezávání KV cache:** Tenzorové uvolňování paměti s udržení klíčových kotev pozornosti (*attention sinks*) na úrovni inference (StreamingLLM (4), H_2O (5)).
- Pozornost a inženýrství promptů:**
- **Context Rot:** Degradace schopnosti modelu rovnoměrně využívat informace v dlouhém kontextovém okně (jev *Lost in the Middle* (6)).
 - **Multi-Needle Reasoning:** Schopnost současně nalézt a logicky propojit několik na sobě závislých faktů napříč soubory; s rostoucí délkou kontextu prudce klesá.
 - **Systémový prompt:** Základní direktiva definující identitu agenta, dostupné nástroje a mantinely (formát commitů, zákaz destruktivních příkazů) (7).

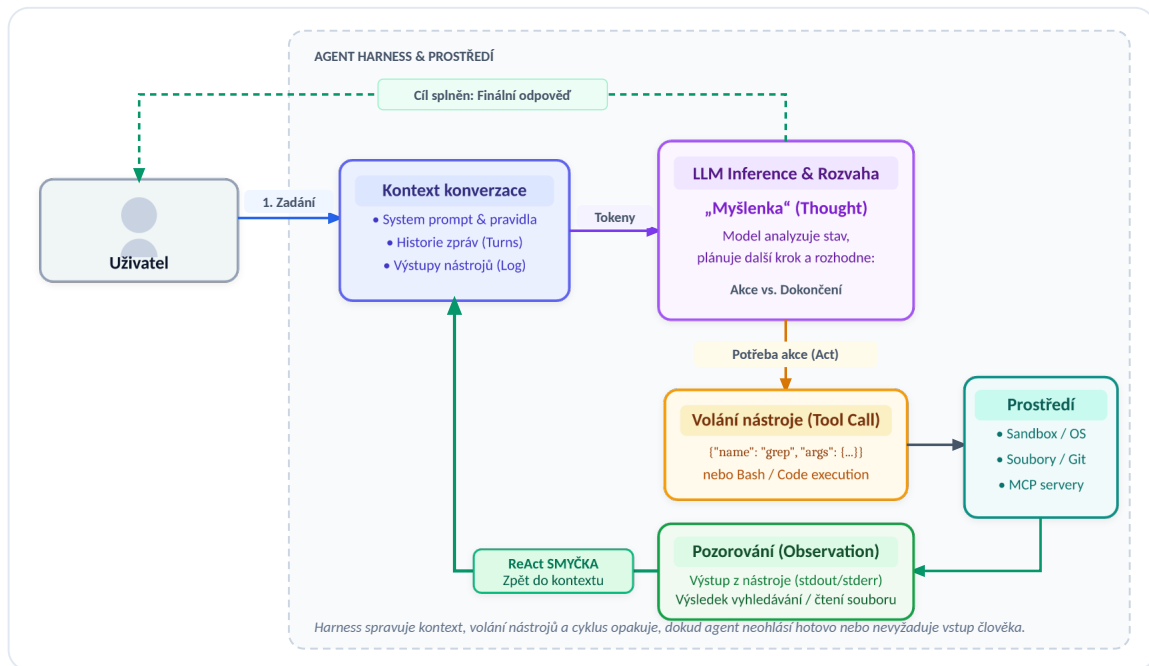
- **Few-shot a Chain-of-Thought:** Vzorové ukázky řešení a vedení modelu k explicitní formulaci mezikroků uvažování před samotným zápisem kódu.
- **Úskalí negativních instrukcí:** Modely často porušují zákazy formulované negací (např. „nemazat existující testy“), protože matice pozornosti (QK^T) asociativně aktivuje zakázané pojmy dříve, než autoregresní proces uplatní logický operátor negace.
- ▶ **Inženýrské řešení:** Afirmativní formulace pravidel (pozitivní vymezení povolených mantinelů) kombinovaná s deterministickou ochranou v harnessu (připojení chráněných souborů pouze pro čtení, blokáce v CI).

💡 **Návrh na vylepšení: Doporučení schématu správy kontextu:** Doporučujeme zařadit srovnávací diagram znázorňující rozdíl mezi destruktivní rekurzivní textovou kompresí (Compaction) a tenzorovým prořezáváním KV cache (StreamingLLM / H_2O) či externím grafem stavu projektu. Schéma pomůže vizualizovat zachování klíčových kotev pozornosti.

2.3 Architektura řídicího harnessu a orchestrace

Základní komponenty řídicího systému:

- **Řídicí harness:** Dozorčí a orchestrační program obklopující inferenční jádro. Spravuje stav sezení, sestavuje systémový prompt, zajišťuje bezpečné provádění nástrojů a řídí iterativní smyčku uvažování.
- **ReAct smyčka (*Reasoning + Acting*)** (8): Čtyřfázový prováděcí cyklus znázorněný na Obrázek 1:
 1. **Rozvaha (*Thought*):** Analýza aktuálního stavu kontextu modelem a formulace nejbližšího záměru.
 2. **Volání nástroje (*Tool Call*):** Emitování strukturovaného požadavku na provedení konkrétní akce.
 3. **Vykonání a pozorování (*Observation*):** Bezpečný běh akce v harnessu a vložení výstupu do kontextu.
 4. **Navazující iterace:** Předložení aktualizovaného kontextu modelu v dalším tahu.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Patologie divergence ReAct smyčky:

- **Perseverance a zacyklení:** Opakované emitování identického volání nástroje se stejnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy. V kontextu vzniká pravděpodobnostní atraktor, který nutí model k opakování chybného vzorce.
- **Oscilace a těkání (Thrashing):** Strídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije A).
- **Nekontrolovaná spotřeba zdrojů (Context Runaway):** Rychlé vyčerpání kontextového okna i rozpočtu na volání API.

Deterministické bezpečnostní pojistky harnessu:

- **Rozpočet tahů a nákladů (Step & Cost Budget):** Pevný limit maximálního počtu tahů $T_{\max}$ (typicky 25–50 kroků) a finanční strop pro tokeny; při překročení dochází k okamžitému zastavení inference.
- **Algoritmická detekce uvíznutí (Stuck Detection):** Výpočet kanonického hashe volání nástroje v čase t :

$$h_t = \text{hash}(\text{nástroj}, \text{canonicalize}(\text{argumenty}))$$

Harness v klouzavém okně posledních k tahů detekuje shodu $h_t = h_{t-1} = \dots = h_{t-k+1}$ nebo cyklické periody.

- **Dvoustupňová intervence:**

- **1. stupeň:** Injekce syntetického varování rozbíjejícího pravděpodobnostní atraktor v matici pozornosti.

- **2. stupeň (Circuit Breaker):** Tvrdé přerušení cyklu, automatický návrat změn v gitu na poslední stabilní commit (`git checkout`) a eskalace člověku.

Způsoby interakce s výkonným prostředím:

- **Strukturované volání nástrojů (Tool / Function Calling):** Validace vstupů a výstupů proti JSON schémátům. Spolehlivé pro atomické operace, ale nese tokenovou režii schémat.
- **Přímé spouštění kódu (Code Execution):** Spouštění generovaných skriptů v sandboxu. Maximální flexibilita, avšak vyžaduje striktní bezpečnostní izolaci.

 **Hlubková kritika / Oponentura: Iluzorní bezpečnost pískoviště:** Přímé spouštění netestovaného syntetického kódu v běžném Docker kontejneru nelze považovat za plnohodnotnou bezpečnostní hranici (*security boundary*). Přístup k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky environmentálních tajností (GitHub tokeny, API klíče k LLM) přes skryté síťové kanály a kompromitaci CI infrastruktury. Pro bezpečný produkční provoz je nezbytná formální izolace na bázi microVM (např. AWS Firecracker, gVisor) a striktní izolace síťových jmenových prostorů.

Modulární rozšiřitelnost harnessu:

- **Dovednosti (Skills):** Adresáře instrukcí a referencí se souborem `SKILL.md` (YAML frontmatter). Do výchozího kontextu se načítají pouze stručná metadata; detailní návod se načítá až při vyvolání nástroje.
- **Skripty a záchytné body (Scripts & Hooks):** Deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.
- **Model Context Protocol (MCP) (9):** Otevřený standard JSON-RPC (přes `stdio` nebo `SSE`) pro sjednocenou integraci externích nástrojů a oddělení běhového prostředí agenta od implementace nástrojů.
- **Meta Harness (10):** Samořízená evoluce a adaptace harnessu samotným agentem. Vhodné pro osobní agenty; pro podnikový vývoj vyžaduje pevně uzamčené bezpečnostní hranice.

Škálování a víceagentní orchestrace:

- **Subagenti (Subagents):** Hierarchická dekompozice úlohy orchestrátorem na specializované agenty (průzkumník repozitáře, plánovač, kódovací dělník). Kontext subagenta je po dokončení zahozen a orchestrátoru je předán pouze čistý výsledek, což chrání primární kontext před znečištěním (*context pollution*).
- **Pracovní postupy jako grafy (DAG / Graph Engineering):** Formalizace fází životního cyklu jako orientovaného acyklického grafu (detekce → plán → kód → testy → schválení). Hrany definují striktní závislosti (`needs`); selhání v libovolném uzlu okamžitě zastaví navazující kroky.

Hlubková kritika / Oponentura: Nekontrolovaná mutace v konceptu Meta Harness:

Povolení samořízené evoluce harnessu samotným agentem (modifikace vlastních instrukcí, pravidel a nástrojů) představuje obrovské bezpečnostní a stabilitní riziko. Pokud agent v iteraci k v důsledku mírné halucinace uvolní bezpečnostní pravidlo nebo oslabí validační podmínku, v iteraci $k + 1$ ji přijme jako normu (uncontained meta-harness mutation). Pro produkční enterprise prostředí je nezbytné, aby řídicí harness obsahoval kryptograficky podepsané, neměnné jádro pravidel (Immutable Policy Core), které agent nesmí za žádných okolností modifikovat.

2.4 Zapojení člověka do smyčky (Human-in-the-loop)

Principy řízeného lidského dohledu nad autonomním vývojem:

- **Princip Human-in-the-loop:** Cílem není nekritická plná autonomie, nýbrž automatizace rutinních a mechanických kroků v kombinaci s lidským rozhodováním tam, kde je změna nevratná nebo kde chybí jednoznačné algoritmické měřítko správnosti.
- **Lidské schvalovací brány (Human Gates):** Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora.
 - ▶ **Dvoufázové schvalování:**
 - **1. brána (Záměr a plán):** Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód.
 - **2. brána (Sémantická revize):** Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
 - ▶ **Prevence únavy z revizí (Review Fatigue):** Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- **Dohledatelnost původního zadání:** Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- **Transparentnost selhání a deterministická eskalace:** Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu tahů nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

 **Hlubková kritika / Oponentura: Kognitivní limity lidské schvalovací brány (Review Fatigue):** Spoléhat se na finální sémantickou kontrolu diffu v pull requestu naráží na lidské kognitivní limity. Výzkumy prokazují, že u rozsáhlých diffů (nad 300–400 řádků) dramaticky klesá hloubka lidské pozornosti — vývojář kód pouze zběžně prohlédne a spoléhá na zelenou fajfku z CI. Aby byla lidská brána efektivní, harness musí diffy rozkládat do sémanticky sevřených mikro-

kroků, generovat interaktivní vysvětlení netriviálních rozhodnutí a explicitně zvýrazňovat změny v kritických architektonických komponentách.

KONCEPT

3 Praktická část

3.1 Cíl a rozsah systému DarkFactory

- **Systém DarkFactory (11):** Agentní harness instalovatelný jako aplikace pro platformu GitHub (GitHub App).
- **Cíl systému:** Maximalizace automatizace rutinních fází softwarového inženýrství:
 - Sémantická analýza požadavků v GitHub Issues.
 - Technické plánování a dekompozice úlohy.
 - Generování a úprava zdrojového kódu.
 - Běh validačních testů a automatické opravy chyb.
 - Otevření strukturovaného pull requestu.
- **Míra autonomie:** Kooperativní model se zapojením člověka (*Human-in-the-loop*). Vzhledem ke stochastické povaze LLM nelze systém označit za plně autonomní; modely nemohou nést konečnou architektonickou odpovědnost.
- **Dělbá rolí:**
 - **Autonomní agent:** Kognitivně rutinní a mechanické operace (průzkum souborů, syntéza kódu, řešení syntaktických regresí, běh testů).
 - **Lidský inženýr:** Vymezení záměru (co a proč stavět), schvalování technických plánů a finální sémantická kontrola kódu v PR.

 **Hlubková kritika / Oponentura: Bezpečnostní perimetr aplikace GitHub App:** Architektura DarkFactory staví na instalaci jako GitHub App s právy zápisu do repozitáře a správy pull requestů. Pro podnikové nasazení to představuje zásadní vektor útoku: pokud model podlehně nepřímému prompt injection útoku (např. ze zlomyslného komentáře v issue či neznámého balíčku), získává přístup k interním tokenům repozitáře. Při obhajobě je nutné explicitně vymezit, jak harness izoluje tajnosti repozitáře, omezuje síťový provoz sandboxu a vynucuje princip nejmenších oprávnění (*least privilege*).

3.2 Architektura a životní cyklus požadavku

 **Strukturální upozornění: Architektura a životní cyklus v rekonstrukci:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory byla dosavadní implementační dekompozice modulů, stavový automat životního cyklu a konfigurační model dočasně nahrazeny zástupnými strukturálními bloky. Nová architektura, komponentní diagram a specifikace rozhraní budou doplněny po stabilizaci nové verze.

- **Architektonická modularita:** Striktní oddělení deklarativního konfiguračního manifestu klientského repozitáře od sdílené orchestrační logiky.
- **Hermetické běhové prostředí:** Izolovaný běh úloh v kontejnerech se zamčenými verzemi závislostí a nástrojů.

- **Životní cyklus požadavku (Stavový automat / DAG):**
 - ▶ **1. Příjem:** Detekce nového zadání v GitHub Issues.
 - ▶ **2. Plánování:** Analýza kódu a návrh kroků implementace.
 - ▶ **3. Lidská brána 1:** Schválení plánu vývojářem.
 - ▶ **4. Implementace:** Autonomní editace souborů v izolované větvi.
 - ▶ **5. Validace:** Běh testů a kontrol v CI s automatickou samoopravou chyb.
 - ▶ **6. Vystavení PR:** Otevření pull requestu s popisem a diffem.
 - ▶ **7. Lidská brána 2:** Finální kontrola a schválení člověkem.
 - ▶ **8. Integrace:** Automatické sloučení do hlavní větve (*squash and merge*).
- **Dvouúrovňová detekce projektů:**
 - ▶ **Prostředí (Environment):** Určuje vyžadované binární nástroje a balíčkovací manažery podle detekovaných souborů (viz Tabulka 1).
 - ▶ **Doména (Domain):** Určuje způsob řízení výstupů (doména programového kódu vs. textová sazba a dokumentace).

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

- **Deklarativní manifest:** Umožňuje automaticky detekované parametry deklarativně přepsat či rozšířit (např. o explicitní cesty k písmům nebo dodatečné integrační kontroly).

 **Návrh na vylepšení:** Placeholder: Zde bude doplněna nová vrstevnatá dekompozice přestavěného systému DarkFactory, stavový diagram životního cyklu požadavku a deklarativní schéma konfiguračního manifestu.

3.3 Orchestrační jádro a validační infrastruktura

 **Strukturální upozornění: Orchestrace a validační subsystém v rekonstrukci:** Mechanismus rotace poskytovatelů LLM, správa tokenů, verifikační pipeline a správa nastavení jako kód procházejí rekonstrukcí podle nového orchestračního enginu DarkFactory.

* Pojem **hermetické prostředí** označuje výpočetní prostředí zcela izolované od nekontrolovaných stavů hostitelského systému a sítě. Veškeré nástroje, knihovny a závislosti jsou explicitně uzamčeny na konkrétních verzích, což zaručuje determinismus a reprodukovatelnost.

Klíčové operační mechanismy orchestračního jádra:

- **Rotace poskytovatelů LLM:** Úkoly jsou definovány nezávisle na formátu konkrétního poskytovatele. Při vyčerpání kvóty nebo výpadku API primárního modelu harness automaticky přepne na navazující model v rotačním žebříčku, což eliminuje prostoje pipeline.
- **Deterministické ověřování změn:** Každý zásah do kódu podléhá požadovaným kontrolám v CI. Úlohy končí explicitním stavem, čímž brání zablokování větve v důsledku přeskočených kontrol.
- **Správa nastavení jako kód (*Config as Code*):** Pravidla ochrany větví, konfigurace linterů i projektové nástěnky (GitHub Projects) jsou definovány deklarativně a podléhají verzování v gitu.
- **Automaticky odvozovaná živá dokumentace (*Living Documentation*):** Dokumentace a specifikace jsou generovány přímo ze zdrojového kódu a strukturovaných inline komentářů s automatickou validací v CI.

 **Hlubková kritika / Oponentura: Asymetrie formátů a ztráta kontextu při rotaci poskytovatelů:** Rotace modelů různých poskytovatelů (např. přepnutí z Claude na GPT či DeepSeek při vyčerpání kvóty) naráží na zásadní odlišnosti v tokenizérech, syntaxi nástrojů (XML vs. JSON schemas) a vnímání systémového promptu. Při náhlém předání kontextu hrozí ztráta návaznosti v rozpracovaném plánu. Text musí vysvětlit přítomnost normalizační vrstvy (provider adapter), která udržuje historii tahů a výpisy nástrojů v neutrální kanonické reprezentaci.

 **Návrh na vylepšení:** Placeholder: Zde bude doplněn detailní popis nového modelu rotace poskytovatelů, správy tokenových rozpočtů, cachování závislostí a generování živé dokumentace po dokončení přestavby.

3.4 Systém revizních značek pro lidský dohled nad akademickým textem

Protokol vizuálních revizních značek pro formát Typst formalizuje dohled člověka nad generovaným textem:

1. Postranní revizní panely (*Callouty*):

- `#note[...]` (zelený): Konstruktivní náměty na vylepšení, doplnění diagramů a praktické vazby.
- `#issue[...]` (červený): Věcné nesrovnalosti, logické mezery, překlepy a duplicitní obsah.
- `#alert[...]` / `#struct-alert[...]` (žlutý): Strukturální nevyváženost, chybějící sekce a osnovové neshody.
- `#critique[...]` (oranžový): Hlubková oponentura, slepá místa, zpochybnění neověřených předpokladů k obhajobě.

- `#blue-note[...]` (modrý): Metodické vymezení, rozsah práce a konceptuální mantinely zadání.

💡 **Návrh na vylepšení:** Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

⚠️ **Chyba / Nesrovnalost k opravě:** Detekované věcné nepřesnosti, logické mezery, překlady nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

📐 **Strukturální upozornění:** Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

🔥 **Hloubková kritika / Oponentura:** Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabin formulovaných jako břitké otázky k obhajobě.

📌 **Metodické vymezení / Rozsah práce:** Metodické vymezení a rozsah práce — formulace hranic zkoumaného problému, metodická abstrakce (např. oddělení agentního inženýrství od strojového učení) a mantinely zadání.

2. Textové revizní funkce v toku textu:

- **Neověřený koncept** (`#draft[...]` / `#unconfirmed[...]`): Žluté podbarvení — koncept čekající na posouzení a revizi autorem.
- **Nově přidáný text** (`#added[...]`): Zelené podbarvení — nově vygenerovaný text začleněný agentem.
- **Potvrzený text** (`#confirmed[...]`): Modré podbarvení — text potvrzený uživatelem, čekající na finální integraci.
- **Navrženo k odstranění** (`#removed[...]`): Červené přeškrtnutí — zastaralý text navržený ke smazání.
- **Srovnávací diff** (`#diff(old, new)`): Červený původní text přeškrtnutý následovaný zeleným novým textem.
- **Čistý neoznačený text:** Finální, autorsky schválený text v hlase autora bez podbarvení.

💡 **Návrh na vylepšení: Automatizovaná kontrola revizních značek v CI:** Doporučujeme propojit protokol revizních značek s validační pipeline DarkFactory: CI linter by měl ověřit, že před finálním vydáním nezůstala v rukopisu žádná neuzavřená značka vady (`#issue`) a že publikovaná čistá verze `prace.pdf` deterministicky potlačuje všechny interní callouty a diffy.

4 Výsledky a diskuse

📌 **Metodické vymezení / Rozsah práce: Metodické vymezení výsledků a diskuse:** V souladu se zaměřením práce na principy agentního inženýrství a architekturu řídicího harnessu se tato kapitola soustředí na systémové a architektonické vlastnosti navrženého řešení: modularitu, determinismus, bezpečnostní izolaci a srovnání s existujícími přístupy (monolitické agentní smyčky vs. řízené DAG workflow).

4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami

Architektonické principy centralizovaného harnessu:

- **Centralizace pracovních postupů:** Klientské repozitáře neudrží vlastní izolované skripty; exekuci delegují na centrální znovupoužitelné šablony a specifičnost projektu vymezují v deklarativním manifestu.
- **Údržbová složitost $O(1)$:** Bezpečnostní oprava či aktualizace v harnessu je provedena jednou a spotřebitelské projekty ji přebírají posunem připnuté verze ($O(1)$ oproti $O(N)$ manuálním úpravám v každém repozitáři zvlášť).
- **Statická kontrola jádra:** Jádro harnessu podléhá striktní typové kontrole a jednotkovému testování před každým vydáním.
- **Omezení jediného bodu selhání (SPOF):** Projekty odkazují na neměnný kryptografický SHA hash commitu či sémantický tag, což vylučuje nechtěné regresní změny.
- **Konzistentní agentní prostředí:** Jednotný životní cyklus požadavků, správa kontextu, rotace modelů i vyhodnocení validačních bran napříč všemi projekty.
- **Oddělení prostředí od domény:**
 - ▶ **Doména programového kódu:** Kompilace, statická analýza, jednotkové testy a měření pokrytí.
 - ▶ **Doména textu a dokumentace:** Sazební kompilace, kontrola terminologie a generování auditovatelných PDF artefaktů.
- **Univerzální orchestrační automat:** Řídicí harness obsluhuje obě domény shodným stavovým automatem bez větvení integrační logiky.

🔥 **Hlubková kritika / Oponentura: Riziko kaskádového rozpadu při centralizaci šablon:** Argumentace údržbovou složitostí $O(1)$ zamlčuje druhou stranu mince: chybná nebo nekompatibilní změna v centrálním harnessu okamžitě ohrožuje buildy ve všech klientských repozitářích (kaskádové selhání upstreamu). V textu je nezbytné obhájit, proč je absolutně vyžadováno striktní připínání na konkrétní neměnný SHA hash commitu, a jaké postupy postupného nasazování (canary testing) harness využívá pro ověření změn před jejich plošným nasazením.

4.2 Provozní metriky a spolehlivost agentních běhů

🔧 **Strukturální upozornění: Metriky čekají na novou evaluaci:** Původní ad-hoc měření byla vyřazena. Po dokončení a stabilizaci nové verze harnessu proběhne série strukturovaných měření nad reálnými vývojovými požadavky.

💡 **Návrh na vylepšení:** Placeholder: Zde bude zařazena nová tabulka empirických metrik (počet zpracovaných požadavků, úspěšnost PR na první pokus, úspěšnost po automatické samoopravě, průměrná spotřeba tokenů na tah, časová latence a spolehlivost rotace modelů).

4.3 Poznatky a systémová úskalí z provozu

Poznatky z nasazení autonomních agentů v CI:

- **Řízení souběžnosti a větvené zámky:** Paralelní integrační běhy vyžadují striktní zámky na úrovni větví, aby nedocházelo ke kolizím a uvážnutí workflow.
- **Transparentnost hlášení selhání:** Zákaz tichého pohlcování výjimek; každá chyba nástroje nebo API musí být zaznamenána do kontextu a eskalována člověku. Tiché maskování chyb vede k masivním halucinacím modelu.
- **Deklarativní detekce prostředí:** Vyloučení implicitních předpokladů o repozitáři; veškeré kroky se deterministicky odvozují z přítomnosti souborů a deklarativního manifestu.

4.4 Diskuse: Porovnání architektur

Porovnání navrženého harnessu se současnými platformami (SWE-agent (8), Devin, Copilot Workspace):

- **Deterministický DAG vs. nekonečná smyčka:**
 - **Volné smyčky (SWE-agent):** Monolitický model v interaktivním terminálu sám rozhoduje o ukončení; při uvíznutí v atraktoru snadno vyčerpá rozpočet tokenů.
 - **Navržený harness:** Model je uzavřen do deterministického grafu (DAG) se striktně alokovaným rozpočtem tahů na každou fázi a izolací kontextu.
- **Granularita lidského dohledu:**
 - **Jednorázový přístup:** Přímé vygenerování celého PR; při špatném pochopení zadání vede ke stovkám řádků vadného kódu a únavě vývojáře z revizí (*review fatigue*).
 - **Dvoufázové schvalování:** Člověk autorizuje záměr a technický plán dříve, než agent začne zasahovat do repozitáře.
- **Bezpečnost a hermetičnost:**
 - **Lokální stroje:** Riziko poškození prostředí či úniku tajemství při spouštění netestovaného kódu.

- ▶ **Efemérní CI kontejnery:** Běh v izolovaném sandboxu s minimálními oprávněními tokenů a auditovatelným protokolem všech operací.

💡 **Návrh na vylepšení: Doporučení empirického srovnání se SWE-bench:** Pro rigorózní akademické zhodnocení doporučujeme po stabilizaci nového jádra DarkFactory podrobit harness standardnímu benchmarku SWE-bench Lite. Porovnání úspěšnosti (`Pass@1`) a spotřeby tokenů vůči referenčním systémům (SWE-agent, AutoCodeRover) poskytne nezpochybnitelný empirický důkaz funkčnosti navržených bezpečnostních pojištění.

4.5 Systémová a metodická omezení

Limity navržené architektury:

- **1. Hranice deterministické verifikovatelnosti:** Autonomie je spolehlivá pouze u objektivně testovatelných změn (kód, testy, typy). U subjektivních úloh (UX ergonomie, grafický design, stylistická formulace textu) zůstává role modelu asistenční a validaci provádí člověk.
- **2. Propustnost a dostupnost inferenčních API:** Rotační žebříček tlumí lokální výpadky, avšak globální kvóty a latence cloudových poskytovatelů tvoří pevný strop průchodnosti pipeline.
- **3. Absence vícevláknového řešení konfliktů:** Souběžná práce mnoha autonomních agentů a lidí nad týmiž soubory a řešení komplexních merge konfliktů představuje otevřenou výzvu pro navazující výzkum.

🔥 **Hlubková kritika / Oponentura: Metodologická závislost na kvalitě testovací sady repozitáře:** Celá premisa deterministického ověřování předpokládá, že repozitář disponuje kvalitní a přísnou testovací sadou. V praxi je však pokrytí kódu testy často nízké nebo testy ověřují pouze triviální scénáře. Pokud agent provede změnu, která projde chudou sadou testů, ale zavleče subtilní architektonickou vadu, deterministická brána selhává. V diskusi je nutné rozebrat pokročilé techniky verifikace, např. mutační testování (*mutation testing*) či generování regresních testů agentem před samotným zásahem do produkčního kódu.

5 Závěr

📌 **Metodické vymezení / Rozsah práce: Poznámka k vypracování závěru:** Závěr práce bude sepsán jako poslední krok po definitivním ucelení teoretických východisek agentního inženýrství a detailní architektury řídicího harnessu. Tato závěrečná kapitola syntetizuje zjištění o deterministickém řízení autonomních agentů a zhodnotí formulované principy a výzkumné otázky bez vazby na dřívější ad-hoc testovací repozitáře.

KONCEPT

Seznam příloh

A	Obsah přiloženého média	25
B	Schéma konfiguračního manifestu darkfactory.json	26
C	Sdílené workflow pro GitHub Actions	27
D	Systémové prompty plánovacího a kódovacího agenta	28
E	Protokol revizních značek v sazebním systému Typst	29

KONCEPT

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib               # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/              # Sdílené šablony kontinuální
integrace
│   └─ src/                            # Výkonné orchestrační jádro systému
├─ out/prace.pdf                       # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

 **Strukturální upozornění: Příloha v rekonstrukci:** Formální JSON schéma manifestu bude doplněno po stabilizaci nového schématu v přestavěném systému DarkFactory.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedeno úplné JSON Schema vymezující validní syntaxi nového konfiguračního manifestu.

KONCEPT

C Sdílené workflow pro GitHub Actions

 **Strukturální upozornění: Příloha v rekonstrukci:** Definice sdílených a volajících workflow pro GitHub Actions budou aktualizována po dokončení přestavby orchestrátoru.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny ukázky znovupoužitelného a volajícího workflow nové verze DarkFactory.

KONCEPT

D Systémové prompty plánovacího a kódovacího agenta

 **Strukturální upozornění: Příloha v rekonstrukci:** Systémové prompty pro plánovací a implementační fáze budou aktualizovány podle nových rolí a instrukčních šablon v přestavěném systému.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny kompletní systémové prompty pro jednotlivé fáze životního cyklu požadavku.

KONCEPT

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břitká, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabin metodiky a příprava na otázky zkušební komise.

 **Metodické vymezení / Rozsah práce:** Ukázka modrého panelu metodického vymezení (`#blue-note` / `#scope-note`): Vymezení rozsahu práce, formulace mantinelů a architektonických abstrakcí.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** (`#diff`): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text:** Představuje finální, autorsky schválený text v hlase autora bez jakéhokoli podbarvení.

Role značek v lidském dohledu:

- **Čistý neoznačený text:** Schválený autorský text v hlase autora.
- **Žluté podbarvení** (`#unconfirmed`): Neověřený koncept čekající na lidské posouzení.
- **Zelené podbarvení** (`#added`): Nově vygenerované návrhy agenta.
- **Modré podbarvení** (`#confirmed`): Uživatelem odsouhlasený text.
- **Srovnávací diff** (`#diff`): Transparentní vizualizace navržených oprav.
- **Postranní panely (Callouty):** Striktní oddělení námětů, chyb a oponentury od těla textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. XIAO, Guangxuan, TIAN, Yuandong, CHEN, Beidi, HAN, Song a LEWIS, Mike. Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*. Online. 2023. Available from: <https://arxiv.org/abs/2309.17453>
5. ZHANG, Zhenyu, SHENG, Ying, ZHOU, Tianyi, CHEN, Tianlong, ZHENG, Lianmin, CAI, Ruisi, SONG, Zhao, TIAN, Yuandong, RÉ, Christopher, BARRETT, Clark, WANG, Zhangyang a CHEN, Beidi. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 34661–34730. Available from: <https://arxiv.org/abs/2306.14048>
6. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
10. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
11. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
12. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>

13. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
14. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>

KONCEPT